

Experiment Name

Write a program to identify unique identifiers from a C program using a basic lexical analyzer.

Objective

The objective of this experiment is to design and implement a basic lexical analyzer that scans a C program and detects valid identifiers. The program identifies unique identifiers while ignoring keywords, invalid tokens, and duplicates. This helps in understanding the working of lexical analysis in compiler design.

Algorithm

1. Start the program.
2. Read the input C program from a file or user input.
3. Define a list of C keywords (such as int, float, char, if, while, return, etc.).
4. Initialize an empty set or list to store unique identifiers.
5. Read the program line by line.
6. For each line, break the line into tokens using delimiters such as spaces, commas, semicolons, parentheses, and operators.
7. For each token:
 - (a) Check if the token starts with a letter (a–z or A–Z) or an underscore (_).
 - (b) Check if the remaining characters of the token contain only letters, digits, or underscores.
 - (c) If the token does not satisfy identifier rules, skip it.
 - (d) Compare the token with the list of C keywords.
 - (e) If the token matches a keyword, ignore it.
 - (f) Otherwise, check whether the identifier is already stored.
 - (g) If it is not present, add it to the list of unique identifiers.
8. Continue scanning until the entire program is processed.
9. Display all unique identifiers found in the C program.
10. Stop the program.

Source Code

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  bool isKeyword(const string &word)
4  {
5      static const vector<string> keywords = {
6
7          "auto", "break", "case", "char", "const", "continue", "default", "do", "double",
8
9          "else", "enum", "extern", "float", "for", "goto", "if", "int", "long", "register",
10         "return", "short", "signed", "sizeof", "static", "struct", "switch", "typedef",
11         "union", "unsigned", "void", "volatile", "while"
12     };
13
14     for (const auto &k : keywords)
15         if (word == k)
16             return true;
17
18     return false;
19 }
20
21 bool isIgnored(const string &word)
22 {
23     static const vector<string> ignored = {
24         "include", "stdio", "iostream", "printf", "scanf",
25         "cin", "cout", "endl"
26     };
27
28     for (const auto &w : ignored)
29         if (word == w)
30             return true;
31
32     return false;
33 }
34
35 bool isValidIdentifier(const string &word)
36 {
37     if (word.empty() || !(isalpha(word[0]) || word[0] == '_'))
38         return false;
39
40     for (char c : word)
41         if (!(isalnum(c) || c == '_'))
42             return false;
43
44     return true;
45 }
46
47 int main()
48 {
49     ifstream file("input.c");
50     map<string, int> frequency;
51     string word;
52
53     while (file >> word)
54     {
55         word = cleanToken(word);
56
57         if (isValidIdentifier(word) &&
58             !isKeyword(word) &&
59             !isIgnored(word))
60         {
61             frequency[word]++;
62         }
63     }
64
65     for (const auto &entry : frequency)
66     {
67         cout << left << setw(20)
68              << entry.first
69              << entry.second << "\n";
70     }
71
72     return 0;
73 }
```

Figure 1: Source code for lexical analyzer to detect unique identifiers

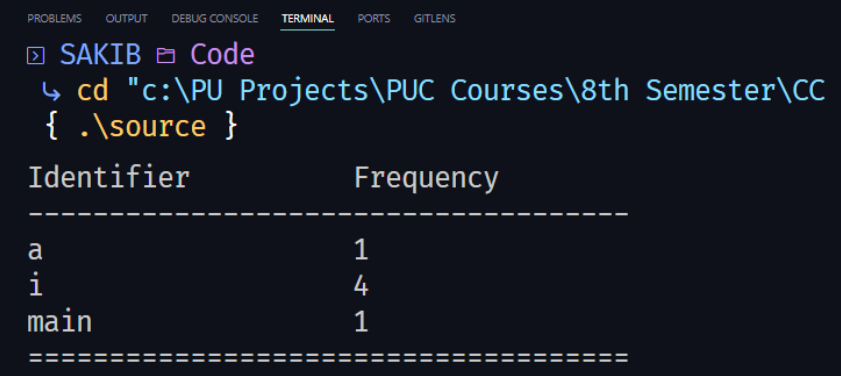
```

1  #include <stdio.h>
2
3  int main()
4  {
5      int i;
6      int a[10];
7      for(i = 1; i <= 10; i++)
8      {
9          printf("Bangladesh\n");
10     }
11
12     return 0;
13 }

```

Figure 2: Input code for lexical analyzer to detect unique identifiers

Output Screenshot



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
SAKIB Code
cd "c:\PU Projects\PUC Courses\8th Semester\CC"
{ .\source }

Identifier      Frequency
-----
a               1
i               4
main            1
=====

```

Figure 3: Output showing all unique identifiers extracted from the C program

Discussion

This experiment demonstrates the design of a simple lexical analyzer to extract identifiers from a C program. The analyzer scans the input, applies identifier rules, removes keywords, and ensures uniqueness using a suitable data structure. This experiment provides a clear understanding of tokenization, identifier recognition, and the importance of lexical analysis in the first phase of a compiler.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Lab Report 06: Write a program to detect and remove left recursion
from a given context-free grammar using C++

Submitted by:

Name	Mohammad Hafizur Rahman Sakib
ID	0222210005101118
Section	C
Session	Fall 2025 (8th Semester)
Date of performance	11.03.2026
Submission Date	31.03.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks

Experiment Name

Write a program to detect and remove left recursion from a given context-free grammar using C.

Objective

The objective of this experiment is to design and implement a C program that takes a context-free grammar as input, detects left recursion in the productions, and removes it by transforming the grammar into an equivalent non-left-recursive grammar. This helps in understanding the importance of eliminating left recursion before applying top-down parsing techniques in compiler design.

Algorithm

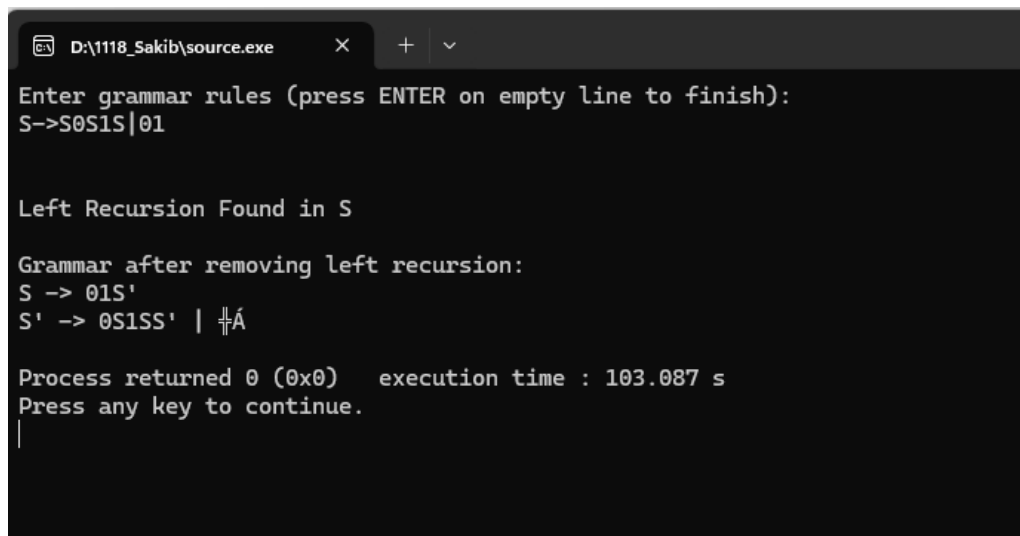
1. Start the program.
2. Read the number of grammar productions or read rules until an empty line.
3. For each production of the form $A \rightarrow \alpha$:
 - (a) Extract the left-hand side (non-terminal) and right-hand side.
 - (b) Split the right-hand side into individual productions using '—' as delimiter.
 - (c) Separate the productions into two groups:
 - Those starting with the same non-terminal A (left recursive - α part).
 - Those not starting with A (β part).
 - (d) If left recursive productions exist:
 - Print the original production and indicate left recursion is found.
 - Transform the grammar using the standard left recursion removal rule:
$$A \rightarrow \beta A' \mid \beta$$
$$A' \rightarrow \alpha A' \mid \epsilon$$
 - (e) If no left recursion is found, print the production as it is.
4. Continue until all productions are processed.
5. Stop the program.

Source Code

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX 50
5  #define LEN 100
6
7  void removeLR(char g[MAX][LEN], int n) {
8      for (int i = 0; i < n; i++) {
9          char L[LEN], R[LEN], *p, prod[10][LEN], a[10][LEN], b[10][LEN];
10         int pc = 0, ac = 0, bc = 0;
11
12         sscanf(g[i], "%[^-]>%[^\\n]", L, R);
13
14         p = strtok(R, "|");
15         while (p) { strcpy(prod[pc++], p); p = strtok(NULL, "|"); }
16
17         for (int j = 0; j < pc; j++)
18             if (!strcmp(prod[j], L, strlen(L)))
19                 strcpy(a[ac++], prod[j] + strlen(L));
20             else
21                 strcpy(b[bc++], prod[j]);
22
23         if (ac) {
24             printf("\nLeft Recursion in %s\n", L);
25             for (int j = 0; j < bc; j++)
26                 printf("%s%s'", b[j], L, j < bc - 1 ? " | " : "");
27
28             printf("\n%s' -> ", L);
29             for (int j = 0; j < ac; j++)
30                 printf("%s%s'", a[j], L, j < ac - 1 ? " | " : "");
31
32             printf(" | ε\n");
33         } else {
34             printf("\nNo left recursion: %s\n", g[i]);
35         }
36     }
37 }
38
39 int main() {
40     char g[MAX][LEN];
41     int n = 0;
42
43     printf("Enter grammar (empty line to stop):\n");
44     while (fgets(g[n], LEN, stdin) && g[n][0] != '\n') {
45         g[n][strlen(g[n], "\n")] = 0;
46         n++;
47     }
48
49     removeLR(g, n);
50     return 0;
51 }
```

Figure 1: Source code for detecting and removing left recursion from a given grammar

Output Screenshot



```
D:\1118_Sakib\source.exe  X  +  v
Enter grammar rules (press ENTER on empty line to finish):
S->S0S1S|01

Left Recursion Found in S

Grammar after removing left recursion:
S -> 01S'
S' -> 0S1SS' | $

Process returned 0 (0x0)   execution time : 103.087 s
Press any key to continue.
|
```

Figure 2: Output showing detection and removal of left recursion

Discussion

This experiment demonstrates the implementation of left recursion removal, which is a crucial step in making a grammar suitable for predictive (top-down) parsing. The program correctly identifies left-recursive productions by checking whether any alternative on the right-hand side begins with the same non-terminal as the left-hand side. It then applies the standard transformation to eliminate direct left recursion while preserving the language generated by the grammar.

The use of string tokenization with `strtok()` and careful parsing of productions makes the implementation efficient for laboratory purposes. This experiment provides hands-on understanding of grammar transformation techniques used in the syntax analysis phase of a compiler.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 04: : : Write a program to match whether a given string is generated by a Given pattern or Not

Submitted by:

Name	Sayed Hossain
ID	0222210005101102
Section	C
Session	Fall 2025
Semester	8th Semester
Performance Date	11.03.2026
Submission Date	28.03.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment No:04

Experiment Name

Write a program to match whether a given string is generated by a given pattern or Not.

Objective

This experiment's goal is to create and execute a program that determines whether a given input string matches a particular regular expression pattern. Understanding pattern matching, regular expressions, and their use in lexical analysis and compiler design is aided by this experiment.

Algorithm

1. Start the program.
2. Read the number of test cases.
3. For each test case:
 - (a) Read the regular expression pattern (R).
 - (b) Read the input string (S).
 - (c) Create a regular expression object using the given pattern.
 - (d) Match the input string against the regular expression.
 - (e) If the string fully matches the pattern, display "Match Found".
 - (f) Otherwise, display "No Match".
4. Stop the program.

Source Code

```
1  #include <iostream>
2  #include <regex>
3  using namespace std;
4
5  int main() {
6      int t;
7      string pattern, str;
8
9      // Read number of test cases
10     cout << "Enter number of test cases: ";
11     cin >> t;
12
13     while (t--) {
14         // Read regex pattern
15         cout << "Enter regular expression: ";
16         cin >> pattern;
17
18         // Read input string
19         cout << "Enter string to match: ";
20         cin >> str;
21
22         // Create regex object
23         regex r(pattern);
24
25         // Match string with regex
26         if (regex_match(str, r)) {
27             cout << "Match Found" << endl;
28         } else {
29             cout << "No Match" << endl;
30         }
31     }
32
33     return 0;
34 }
35
```

Figure 1: Source code for string pattern matching using regular expressions

Output

```
1  SAKIB  Code
2  cd "c:\PU Projects\PUC Courses\8th Semester\CC Lab\Lab 04\Code\" ; if ($?) {
  g++ final.cpp -o final } ; if ($?) { .\final }
3  Enter number of test cases: 1
4  Enter regular expression: a*bb
5  Enter string to match: aaabb
6  Match Found
```

Figure 2: Output showing whether the string matches the given pattern

Discussion

This task shows how to use regular expressions to determine whether a given string is produced by a particular pattern. Using built-in regex libraries, the application correctly verifies whole string matches. This method simplifies the recognition of lexical patterns and offers guidance on the implementation of regular expressions in text processing and compiler design.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 05: : : Write a program to identify unique identifiers from a C program using a basic lexical analyzer.

Submitted by:

Name	Sayed Hossain
ID	0222210005101102
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment Name

Write a program to identify unique identifiers from a C program using a basic lexical analyzer

Objective

This experiment aims to create and build a simple lexical analyser that finds valid identifiers by scanning a C program. While ignoring keywords, incorrect tokens, and duplication, the algorithm finds unique identifiers. This aids in comprehending how lexical analysis functions in compiler design.

Algorithm

1. Start the program.
2. Read the input C program from a file or user input.
3. Define a list of C keywords (such as int, float, char, if, while, return, etc.).
4. Initialize an empty set or list to store unique identifiers.
5. Read the program line by line.
6. For each line, break the line into tokens using delimiters such as spaces, commas, semicolons, parentheses, and operators.
7. For each token:
 - (a) Check if the token starts with a letter (a–z or A–Z) or an underscore ().
 - (b) Check if the remaining characters of the token contain only letters, digits, or underscores.
 - (c) If the token does not satisfy identifier rules, skip it.
 - (d) Compare the token with the list of C keywords.
 - (e) If the token matches a keyword, ignore it.
 - (f) Otherwise, check whether the identifier is already stored.
 - (g) If it is not present, add it to the list of unique identifiers.
8. Continue scanning until the entire program is processed.
9. Display all unique identifiers found in the C program.
10. Stop the program.

Source Code

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  bool isKeyword(const string &word)
4  {
5      static const vector<string> keywords = {
6
7          "auto", "break", "case", "char", "const", "continue", "default", "do", "double",
8          "else", "enum", "extern", "float", "for", "goto", "if", "int", "long", "register",
9          "return", "short", "signed", "sizeof", "static", "struct", "switch", "typedef",
10         "union", "unsigned", "void", "volatile", "while"
11     };
12
13     for (const auto &k : keywords)
14         if (word == k)
15             return true;
16
17     return false;
18 }
19 bool isIgnored(const string &word)
20 {
21     static const vector<string> ignored = {
22         "include", "stdio", "iostream", "printf", "scanf",
23         "cin", "cout", "endl"
24     };
25
26     for (const auto &w : ignored)
27         if (word == w)
28             return true;
29
30     return false;
31 }
32 bool isValidIdentifier(const string &word)
33 {
34     if (word.empty() || !(isalpha(word[0]) || word[0] == '_'))
35         return false;
36
37     for (char c : word)
38         if (!(isalnum(c) || c == '_'))
39             return false;
40
41     return true;
42 }
43 int main()
44 {
45     ifstream file("input.c");
46     map<string, int> frequency;
47     string word;
48
49     while (file >> word)
50     {
51         word = cleanToken(word);
52
53         if (isValidIdentifier(word) &&
54             !isKeyword(word) &&
55             !isIgnored(word))
56         {
57             frequency[word]++;
58         }
59     }
60     for (const auto &entry : frequency)
61     {
62         cout << left << setw(20)
63              << entry.first
64              << entry.second << "\n";
65     }
66
67     return 0;
68 }
69

```

Figure 1: Source code for lexical analyzer to detect unique identifiers

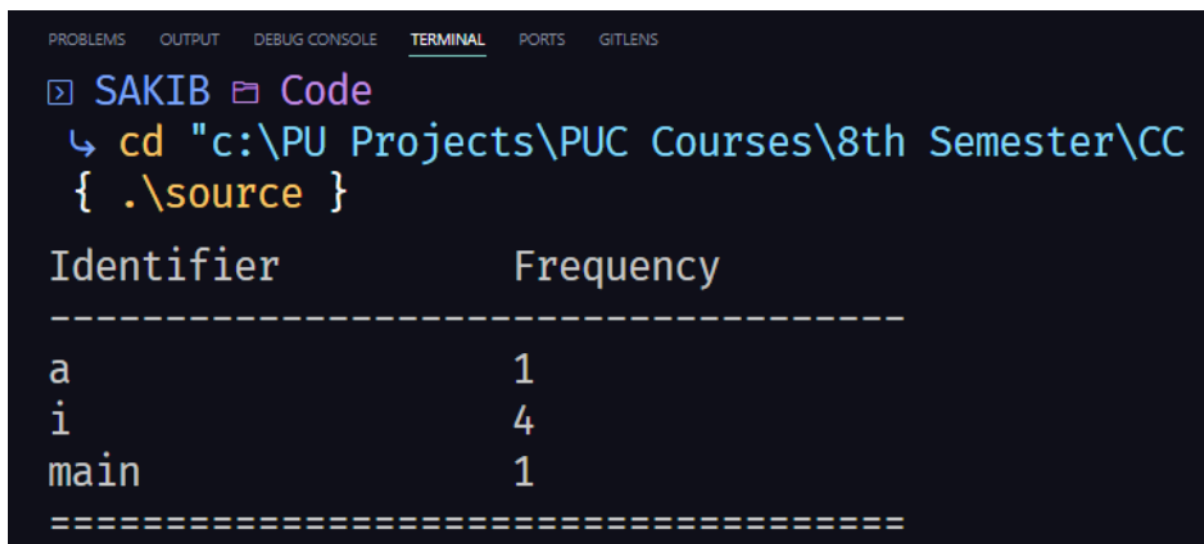
```

1  #include <stdio.h>
2
3  int main()
4  {
5      int i;
6      int a[10];
7      for(i = 1; i <= 10; i++)
8      {
9          printf("Bangladesh\n");
10     }
11
12     return 0;
13 }

```

Figure 2: Input code for lexical analyzer to detect unique identifiers

Output



Identifier	Frequency
a	1
i	4
main	1

Figure 3: Output showing all unique identifiers extracted from the C program

Discussion

The design of a basic lexical analyser to extract identifiers from a C program is demonstrated in this project. After scanning the input, the analyser applies identification rules, eliminates keywords, and uses an appropriate data structure to guarantee uniqueness. Tokenisation, identifier recognition, and the significance of lexical analysis in the initial stage of a compiler are all clearly explained by this experiment.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 06: : : Write a program to detect and remove left recursion from a given context-free grammar using C.

Submitted by:

Name	Sayed Hossain
ID	0222210005101102
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment Name

Write a program to detect and remove left recursion from a given context-free grammar using C.

Objective

This experiment's goal is to create and construct a C program that takes a context-free grammar as input, looks for left recursion in the products, and eliminates it by changing the grammar into a non-left-recursive counterpart. This aids in comprehending the significance of removing left recursion prior to using top-down parsing strategies in compiler design.

Algorithm

1. Start the program.
2. Read the number of grammar productions or read rules until an empty line.
3. For each production of the form $A \rightarrow \alpha$:
 - (a) Extract the left-hand side (non-terminal) and right-hand side.
 - (b) Split the right-hand side into individual productions using '—' as delimiter.
 - (c) Separate the productions into two groups:
 - Those starting with the same non-terminal A (left recursive - α part).
 - Those not starting with A (β part).
 - (d) If left recursive productions exist:
 - Print the original production and indicate left recursion is found.
 - Transform the grammar using the standard left recursion removal rule:
$$\begin{array}{lcl} A \rightarrow \beta A' & | & \beta \\ A' \rightarrow \alpha A & | & \epsilon \end{array}$$
 - (e) If no left recursion is found, print the production as it is.
4. Continue until all productions are processed.
5. Stop the program.

Source Code

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX 50
5  #define LEN 100
6
7  void removeLR(char g[MAX][LEN], int n) {
8      for (int i = 0; i < n; i++) {
9          char L[LEN], R[LEN], *p, prod[10][LEN], a[10][LEN], b[10][LEN];
10         int pc = 0, ac = 0, bc = 0;
11
12         sscanf(g[i], "%[^-]->%[^\\n]", L, R);
13
14         p = strtok(R, "|");
15         while (p) { strcpy(prod[pc++], p); p = strtok(NULL, "|"); }
16
17         for (int j = 0; j < pc; j++)
18             if (!strcmp(prod[j], L, strlen(L)))
19                 strcpy(a[ac++], prod[j] + strlen(L));
20             else
21                 strcpy(b[bc++], prod[j]);
22
23         if (ac) {
24             printf("\nLeft Recursion in %s\n", L);
25             for (int j = 0; j < bc; j++)
26                 printf("%s%s'", b[j], L, j < bc - 1 ? " | " : "");
27
28             printf("\n%s' -> ", L);
29             for (int j = 0; j < ac; j++)
30                 printf("%s%s'", a[j], L, j < ac - 1 ? " | " : "");
31
32             printf(" | ε\n");
33         } else {
34             printf("\nNo left recursion: %s\n", g[i]);
35         }
36     }
37 }
38
39 int main() {
40     char g[MAX][LEN];
41     int n = 0;
42
43     printf("Enter grammar (empty line to stop):\n");
44     while (fgets(g[n], LEN, stdin) && g[n][0] != '\n') {
45         g[n][strcspn(g[n], "\n")] = 0;
46         n++;
47     }
48
49     removeLR(g, n);
50     return 0;
51 }

```

Figure 1: Source code for detecting and removing left recursion from a given grammar

Output


```
Enter grammar rules (press ENTER on empty line to finish):  
S->S0S1S|01
```

```
Left Recursion Found in S
```

```
Grammar after removing left recursion:
```

```
S -> 01S'
```

```
S' -> 0S1SS' | 01
```

```
Process returned 0 (0x0)   execution time : 103.087 s
```

```
Press any key to continue.
```

```
|
```

Figure 2: Output showing detection and removal of left recursion

Discussion

The implementation of left recursion removal, a critical step in preparing a language for predictive (top-down) parsing, is demonstrated in this experiment. By determining if any alternative on the right side starts with the same non-terminal as the left-hand side, the software accurately detects left-recursive productions. The language produced by the grammar is then preserved but direct left recursion is eliminated by applying the standard transformation. The implementation is effective for laboratory use because string tokenisation using `strtok()` and careful parsing of productions are used. This exercise gives participants a practical grasp of the grammar transformation methods employed in a compiler's syntax analysis stage.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 07: : : Write a program to apply Algorithm for Left Factoring a Context-Free Grammar (CFG).

Submitted by:

Name	Sayed Hossain
ID	0222210005101102
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment Name:

Write a program to apply Algorithm for Left Factoring a Context-Free Grammar (CFG).

Objective

Implementing a software that does left factoring on a given Context-Free Grammar (CFG) is the aim of this experiment. One grammar modification method for getting rid of common prefixes is left factoring from productions. By eliminating uncertainty in production options, this transformation helps make the grammar appropriate for top-down parsing techniques like recursive descent parsing.

Algorithm

Step 1: Include required headers and declare variables for grammar, LHS, and RHS productions.

Step 2: Take grammar input (e.g., $A \rightarrow abC|abD|aE$).

Step 3: Extract the left-hand side (non-terminal).

Step 4: Remove LHS and arrow to get the right-hand side.

Step 5: Split RHS using | into individual productions.

Step 6: Store productions in a list/vector.

Step 7: Check for common prefix among productions.

Step 8: If found, separate prefix (α) and suffixes (β).

Step 9: Apply left factoring:

- $A \rightarrow \alpha A'$
- $A' \rightarrow \beta_1 | \beta_2 | \dots | \beta$

Step 10: Replace empty suffix with ϵ .

Step 11: Display the new grammar.

Step 12: If no prefix exists, show “No left factoring needed.”

Step 13: End program.

Source Code:

```
#include <iostream>
#include <vector>
using namespace std;

string prefix(string a, string b){
    string p="";
    for(int i=0; i<min(a.size(), b.size()) && a[i]==b[i]; i++)
        p+=a[i];
    return p;
}

int main(){
    string g, t="";
    cout<<"Enter production (like: A->abC|abD|aE): ";
    cin>>g;

    char A=g[0];
    vector<string> r;

    for(int i=3; i<g.size(); i++){
        if(g[i]=='|'){ r.push_back(t); t=""; }
        else t+=g[i];
    }
    r.push_back(t);

    string p=r[0];
    for(int i=1; i<r.size(); i++)
        p=prefix(p, r[i]);

    if(p==""){ cout<<"No Left Factoring needed\n"; return 0; }

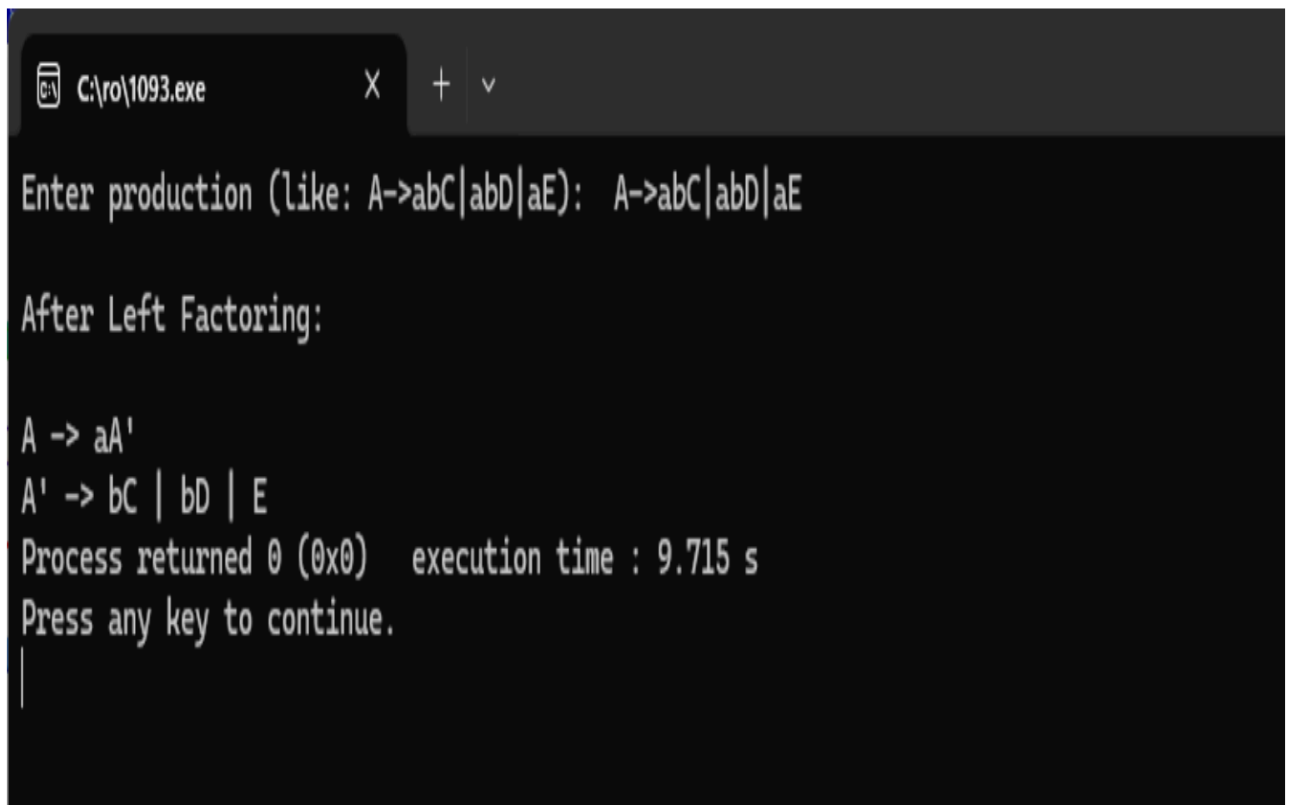
    cout<<"\nAfter Left Factoring:\n\n";
    cout<<A<<" -> " << p << A << "'";

    for(string s:r)
        if(s.substr(0, p.size())!=p)
            cout<<" | " << s;

    cout<<"\n" << A << "' -> ";

    for(int i=0; i<r.size(); i++){
        string suf=r[i].substr(p.size());
        cout<<(suf=="?"?"ε":suf);
        if(i!=r.size()-1) cout<<" | ";
    }
}
```

Output:



```
C:\ro\1093.exe X + v
Enter production (like: A->abC|abD|aE): A->abC|abD|aE

After Left Factoring:

A -> aA'
A' -> bC | bD | E
Process returned 0 (0x0) execution time : 9.715 s
Press any key to continue.
|
```

Discussion

The left factoring technique used in compiler design is demonstrated in this experiment. Because it eliminates ambiguity in grammatical productions that share a prefix, left factoring is crucial. Predictive parsers find it challenging to decide which production rule to apply during parsing because of these ambiguities. The grammar is improved for top-down parsing methods like recursive descent parsing by factoring away the common prefix and adding a new non-terminal symbol. Students learn about grammar transformation and the function of left factoring in syntax analysis through this experiment.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 05: A Program to Identify Unique Identifiers from a C Program and Count Their Frequency.

Submitted by:

Name	Shuvra Roy
ID	02222100051011093
Section	C
Session	Fall 2025
Semester	8th Semester
Performance Date	14.02.2026
Submission Date	28.02.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment No: 05

Experiment Name: A Program to Identify Unique Identifiers from a C Program and Count Their Frequency.

Objective: The objective of this experiment is to design and implement a basic lexical analyzer that can detect identifiers from a given C source program and count the frequency of each identifier. The program follows identifier rules such as allowing letters, digits, and underscores, ensuring the first character is a letter or underscore, and excluding C keywords from identifiers. This experiment helps in understanding token recognition and lexical analysis in compiler design.

Algorithm:

Step 1: Start the program by including the necessary header files such as `iostream`, `fstream`, `map`, and `cctype`. Declare all required variables including file pointers, strings for storing file names and lines, and a map data structure to store identifiers along with their frequency. Also prepare a predefined list of C language keywords so that reserved words are not considered as identifiers during scanning.

Step 2: Initialize an empty map to store identifiers and their frequency count. This data structure allows storing each unique identifier as a key and automatically updating its occurrence count whenever the identifier appears again in the source program.

Step 3: Begin reading the source file line by line using a loop until the end of the file is reached. This ensures that the entire source code is scanned systematically without missing any part of the program.

Step 4: Initialize a boolean flag to detect string literals and a temporary string variable to build words from characters. This flag helps ignore characters that appear inside double quotation marks so that identifiers present inside strings are not incorrectly counted.

Step 5: If the character is a letter, digit, or underscore (_), append it to the temporary word variable. This step helps construct possible identifiers from consecutive valid characters.

Step 6: If a space, punctuation mark, or special symbol is encountered, treat the currently built sequence of characters as a complete word. Then check whether this word satisfies the rules of a valid identifier.

Step 7: Compare the valid word with the list of C language keywords. If the word matches any keyword such as int, while, or return, skip it because keywords cannot be identifiers.

Step 8: If the word is a valid identifier and not a keyword, store it in the map. If the identifier already exists in the map, increase its frequency count by one; otherwise, insert it into the map with an initial frequency of one.

Step 9: Continue this scanning and checking process for every character and every line until the entire source file has been processed completely.

Step 10: After scanning the full source program, write the heading into the output file indicating that the file contains the list of identifiers and their frequencies. Then traverse the map and write each identifier along with its corresponding frequency count in the format:
identifier : frequency.

Step 11: Display the total number of unique identifiers found in the source program on the terminal. Also print a message confirming that the identifier list and frequency information have been successfully saved in the specified output file.

Step 12: Close both the source file and the output file properly to release system resources and avoid file corruption.

Step 13: End the program successfully.

Source Code:

```
1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  #include <cctype>
5
6  using namespace std;
7  set<string> keywords = {
8      "auto", "break", "case", "char", "const", "continue", "default", "do", "double", "else",
9      "enum", "extern", "float", "for", "goto", "if", "int", "long", "register", "return",
10     "short", "signed", "sizeof", "static", "struct", "switch", "typedef", "union",
11     "unsigned", "void", "volatile", "while"
12 };
13 bool isValidIdentifier(const string &word)
14 {
15     if (!(isalpha(word[0]) || word[0] == '_'))
16         return false;
17     for (char c : word)
18         if (!(isalnum(c) || c == '_'))
19             return false;
20     return true;
21 }
22 int main(){
23     string inputFile, outputFile, word;
24     ifstream source;
25     ofstream dest;
26     source.open(inputFile);
27     dest.open(outputFile);
28     if (!source || !dest){
29         cout << "Error opening file!" << endl;
30         return 1;
31     }
32     set<string> identifiers;
33     dest << "List of Unique Identifiers from file (" << inputFile << "):\n\n";
34     while (source >> word){
35         word = cleanWord(word);
36         if (word.empty())
37             continue;
38         if (isValidIdentifier(word) && keywords.find(word) == keywords.end()) {
39             identifiers.insert(word);
40         }
41     }
42     for (const string &id : identifiers)
43         dest << id << endl;
44     source.close();
45     dest.close();
46     cout << "\nTotal Unique Identifiers Found: " << identifiers.size() << endl;
47     cout << "Identifier list saved in: " << outputFile << endl;
48     return 0;
49 }
```

Input: (source.c) file.

```
#include <stdio.h>

int main()
{
    int number = 10;
    int _value = 20;
    int result = number + _value;

    printf("Result = %d\n", result);
    printf("Name: Shuvra Roy ID: 0222210005101093\n");
    return 0;
}
```

Output:

After execution, the program scans the entire C source file and detects all valid identifiers while excluding keywords and invalid tokens. It also counts how many times each identifier appears in the program and writes the result into the output file.

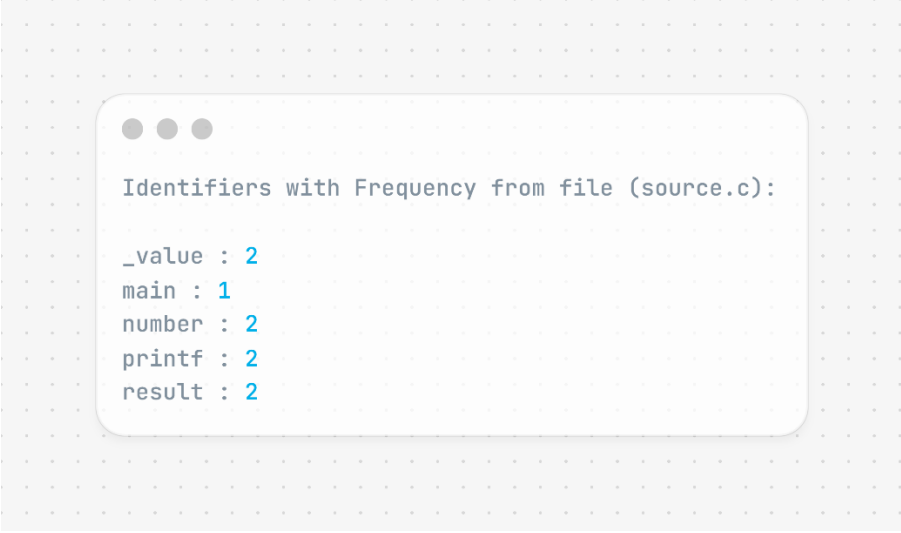
Sample Terminal Output:

```
"E:\8th semester\CCL\lab 5\la  ×  +  v
Enter source C file name: source.c
Enter output file name: output.c

Total Unique Identifiers Found: 5
Identifier frequency saved in: output.c

Process returned 0 (0x0)   execution time : 6.795 s
Press any key to continue.
```

Sample Output File Content: (output.c) file.



```
Identifiers with Frequency from file (source.c):  
  
_value : 2  
main : 1  
number : 2  
printf : 2  
result : 2
```

Discussion:

This experiment demonstrates the implementation of a basic lexical analyzer to detect identifiers in a C program. The program reads the source file, ignores preprocessor directives and string literals, and identifies valid identifiers according to C language rules. It excludes keywords and counts the frequency of each unique identifier using a map data structure. This experiment provides a clear understanding of lexical analysis, token recognition, and identifier classification, which are essential concepts in compiler design and programming language processing.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 06: Write a program to detect and remove left recursion from a given context-free grammar using C.

Submitted by:

Name	Shuvra Roy
ID	02222100051011093
Section	C
Session	Fall 2025
Semester	8th Semester
Performance Date	21.03.2026
Submission Date	28.03.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment No: 06

Experiment Name: Write a program to detect and remove left recursion from a given context-free grammar using C.

Objective: The objective of this experiment is to design and implement a C++ program that detects left recursion in a context-free grammar and removes it by transforming the grammar into an equivalent grammar without left recursion. Eliminating left recursion is important because many top-down parsing techniques used in compiler design cannot handle left-recursive grammars.

Algorithm:

Step 1: Start the program by initializing the necessary variables and including required libraries for input, output, and string processing. Prepare variables to store the grammar production, left-hand side non-terminal, and right-hand side productions.

Step 2: Ask the user to enter a grammar production in the standard format such as $S \rightarrow Sa|b$. The program reads this production as a string and stores it for processing.

Step 3: Extract the left-hand side of the production rule. The first character of the production represents the non-terminal symbol. This symbol will be used to check whether left recursion exists.

Step 4: Extract the right-hand side of the production by removing the left-hand side and the arrow symbol ($\rightarrow$). The remaining part contains one or more productions separated by the $|$ symbol.

Step 5: Split the right-hand side into individual productions using the $|$ symbol as a separator. Each separated string represents a possible production rule.

Step 6: Analyze each production individually. For every production, check the first character of the string to determine whether it is the same as the non-terminal on the left-hand side.

Step 7: Classify the productions into two groups:

- **Left recursive productions (α):** Productions that begin with the same non-terminal symbol.
- **Non-left recursive productions (β):** Productions that begin with a different symbol.

Step 8: Check whether the list of left recursive productions is empty or not.

- If it is empty, then the grammar does not contain left recursion and the program simply prints that no left recursion is found.

Step 9: If left recursion is detected, apply the standard transformation rule used in compiler design to eliminate left recursion. Introduce a new non-terminal symbol (for example A') and transform the grammar according to the following rules:

- $A \rightarrow \beta A'$
- $A' \rightarrow \alpha A' \mid \epsilon$

where β represents the non-left recursive productions and α represents the left recursive parts.

Step 10: Construct the new grammar using the transformation rule and print the grammar without left recursion. Display the result in a clear format so the user can easily understand the transformation.

Step 11: Display both the original grammar and the transformed grammar after removing left recursion to show how the grammar has been modified.

Step 12: End the program after printing the final grammar and releasing any used resources.

Source Code:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
int main()
{
    string production;
    char nonTerminal;
    vector<string> alpha, beta;
    cout << "Enter production (Example: S->Sa|b): ";
    cin >> production;
    nonTerminal = production[0];
    string right = production.substr(3);
    string temp = "";

    for (char c : right){
        if (c == '|'){
            if (temp[0] == nonTerminal)
                alpha.push_back(temp.substr(1));
            else
                beta.push_back(temp);

            temp = "";
        }
    }

    if (alpha.empty()){
        cout << "No left recursion found in the grammar." << endl;
        return 0;
    }

    for (int i = 0; i < alpha.size(); i++){
        cout << nonTerminal << alpha[i];
        cout << " | ";
    }

    cout << endl;
    cout << "\nGrammar after removing left recursion:\n\n";
    cout << nonTerminal << " -> ";

    for (int i = 0; i < beta.size(); i++){
        cout << beta[i] << nonTerminal << " ";
        if (i != beta.size() - 1)
            cout << " | ";
    }

    for (int i = 0; i < alpha.size(); i++){
        cout << alpha[i] << nonTerminal << " ";
        if (i != alpha.size() - 1)
            cout << " | ";
    }

    cout << " | ε" << endl;
    return 0;
}
```

Input and output:

```
"E:\8th semester\CCL\lab 6\Ui" × + ▾
Enter production (Example: S->Sa|b): S->Sa|b
Left recursion detected in the grammar.

Original Grammar:
S -> Sa | b

Grammar after removing left recursion:

S -> bS'
S' -> aS' | ε

Process returned 0 (0x0)   execution time : 1.918 s
Press any key to continue.
```

Discussion: This experiment demonstrates the concept of detecting and removing left recursion in a context-free grammar. Left recursion occurs when a non-terminal symbol appears at the beginning of its own production rule, which can cause infinite recursion in top-down parsers. The program reads a grammar production, checks whether the right-hand side begins with the same non-terminal, and identifies such productions as left recursive. If left recursion exists, the program applies the standard transformation rule to eliminate it by introducing a new non-terminal symbol and rewriting the grammar rules. This transformation preserves the language generated by the grammar while making it suitable for predictive parsing techniques. Through this experiment, students gain practical knowledge of grammar transformation and the importance of removing left recursion in compiler design.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 07: Write a program to apply Algorithm for Left Factoring
a Context-Free Grammar (CFG).

Submitted by:

Name	Shuvra Roy
ID	02222100051011093
Section	C
Session	Fall 2025
Semester	8th Semester
Performance Date	28.03.2026
Submission Date	07.04.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment No: 07

Experiment Name: Write a program to apply Algorithm for Left Factoring a Context-Free Grammar (CFG).

Objective: The objective of this experiment is to implement a program that performs left factoring on a given Context-Free Grammar (CFG). Left factoring is a grammar transformation technique used to eliminate common prefixes from productions. This transformation helps make the grammar suitable for top-down parsing methods such as recursive descent parsing by removing ambiguity in production choices.

Algorithm:

Step 1: Start the program by including the necessary header files and declaring required variables. Prepare variables to store the grammar production, the non-terminal symbol on the left-hand side, and the list of productions on the right-hand side.

Step 2: Ask the user to enter a grammar production in the standard form such as $A \rightarrow abC \mid abD \mid aE$. Read the input as a string and store it for further processing.

Step 3: Extract the left-hand side of the production rule. The first character of the input string represents the non-terminal symbol (for example A). This symbol will remain the main production symbol in the grammar.

Step 4: Extract the right-hand side of the production by removing the non-terminal and the arrow symbol ($\rightarrow$). The remaining part of the string contains multiple productions separated by the $\mid$ symbol.

Step 5: Split the right-hand side into individual production strings using the delimiter $\mid$. Each separated string represents one possible production rule of the grammar.

Step 6: Store all extracted productions into a vector or list data structure. This allows the program to easily process and compare each production.

Step 7: Compare the productions to determine whether a common prefix exists among them. The program examines characters from the beginning of each production and finds the longest sequence of characters that appear at the beginning of multiple productions.

Step 8: If a common prefix is found, store this prefix as α . The remaining part of each production after removing the prefix is considered the suffix and is represented as β .

Step 9: Apply the left factoring rule used in compiler design. Introduce a new non-terminal symbol (for example A') and rewrite the grammar as follows:

- $A \rightarrow \alpha A'$
- $A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

where α represents the common prefix and β represents the remaining parts of the productions.

Step 10: Construct the new grammar rules using the prefix and suffix values. Ensure that empty suffixes are replaced with the epsilon symbol (ϵ) to represent an empty string.

Step 11: Display the transformed grammar clearly so that the user can understand how the original grammar has been modified after applying left factoring.

Step 12: If no common prefix is found among the productions, display a message stating that left factoring is not required for the given grammar.

Step 13: End the program successfully after printing the final grammar rules.

Source Code:

```
● ● ●  
  
#include <iostream>  
#include <vector>  
using namespace std;  
  
string prefix(string a,string b){  
    string p="";  
    for(int i=0;i<min(a.size(),b.size()) && a[i]==b[i];i++)  
        p+=a[i];  
    return p;  
}  
  
int main(){  
    string g,t="";  
    cout<<"Enter production (like: A->abC|abD|aE): ";  
    cin>>g;  
  
    char A=g[0];  
    vector<string> r;  
  
    for(int i=3;i<g.size();i++){  
        if(g[i]=='|'){ r.push_back(t); t=""; }  
        else t+=g[i];  
    }  
    r.push_back(t);  
  
    string p=r[0];  
    for(int i=1;i<r.size();i++)  
        p=prefix(p,r[i]);  
  
    if(p==""){ cout<<"No Left Factoring needed\n"; return 0; }  
  
    cout<<"\nAfter Left Factoring:\n\n";  
    cout<<A<<" -> "<<p<<A<<" ";  
  
    for(string s:r)  
        if(s.substr(0,p.size())!=p)  
            cout<<" | "<<s;  
  
    cout<<"\n"<<A<<"' -> ";  
  
    for(int i=0;i<r.size();i++){  
        string suf=r[i].substr(p.size());  
        cout<<(suf=="?"?"ε":suf);  
        if(i!=r.size()-1) cout<<" | ";  
    }  
}
```

Input:

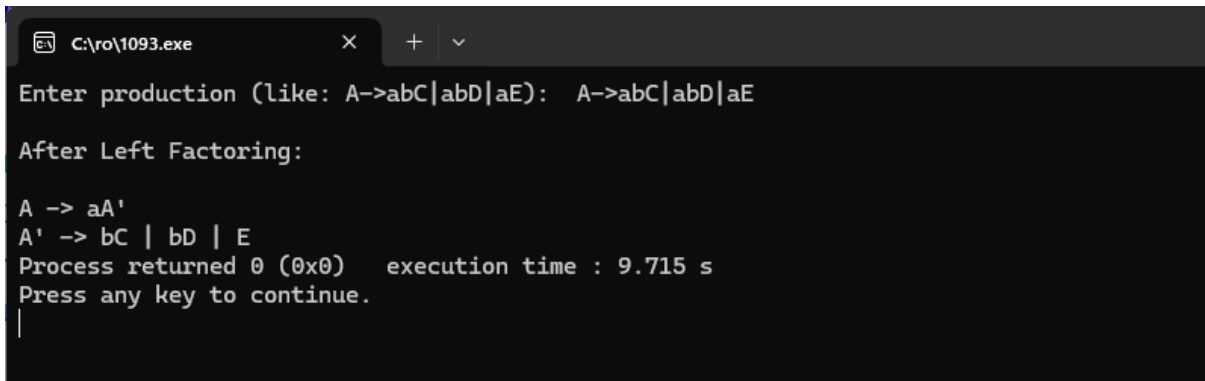
The program takes a Context-Free Grammar (CFG) production from the user. The grammar is entered in the format:

$$A \rightarrow abC \mid abD \mid aE$$

Here, A is the non-terminal and the productions are separated by the symbol |.

Output:

After running the program, it applies the left factoring algorithm to the given grammar and displays the transformed grammar.



```
C:\ro\1093.exe
Enter production (like: A->abC|abD|aE): A->abC|abD|aE

After Left Factoring:

A -> aA'
A' -> bC | bD | E
Process returned 0 (0x0)    execution time : 9.715 s
Press any key to continue.
|
```

This shows the grammar after removing the common prefix.

Discussion:

This experiment demonstrates the implementation of the left factoring technique used in compiler design. Left factoring is important because it removes ambiguity in grammar productions that share a common prefix. Such ambiguities make it difficult for predictive parsers to determine which production rule to use during parsing. By factoring out the common prefix and introducing a new non-terminal symbol, the grammar becomes more suitable for top-down parsing techniques such as recursive descent parsing. Through this experiment, students gain a practical understanding of grammar transformation and the role of left factoring in syntax analysis.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

**Lab Report 01: A program to read a C program
from source file and copy it in a text file**

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 01

Experiment Name

A program to read a source file and copy its contents into a text file using Python.

Objective

The objective of this experiment is to understand file handling operations in Python. It includes opening files, reading content character by character, writing it into a destination file, and proper resource management using context managers.

Algorithm

1. Start the program.
2. Specify source and target file paths.
3. Open source file in read mode and target file in write mode.
4. Read content character by character.
5. Write each character to the target file.
6. Handle errors using exception handling.
7. Display success message.
8. End the program.

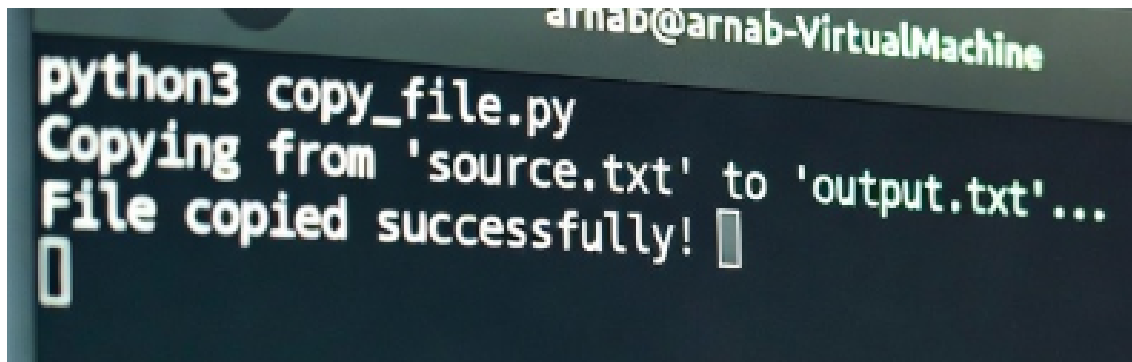
Source Code

```
1 def copy_file(source_path, target_path):
2     try:
3         with open(source_path, 'r', encoding='utf-8') as source:
4             with open(target_path, 'w', encoding='utf-8') as
               target:
5                 while True:
6                     ch = source.read(1)
7                     if not ch:
8                         break
9                     target.write(ch)
10                print("File copied successfully!")
11
12    except FileNotFoundError:
13        print(f"Error: Source file '{source_path}' not found.")
14    except PermissionError:
15        print("Error: Permission denied to access file.")
16    except Exception as e:
17        print(f"Unexpected error: {e}")
18
19 # Main execution
20 if __name__ == "__main__":
21     source_file = "source.txt"
22     target_file = "output.txt"
23
24     print(f"Copying from '{source_file}' to '{target_file}'...")
25     copy_file(source_file, target_file)
```


Output

The program successfully copies the content of `source.txt` into `output.txt`.

Terminal Input / Output Screenshot:

A terminal window with a dark background and light green text. The prompt is 'arnab@arnab-VirtualMachine'. The command 'python3 copy_file.py' has been entered. The output shows 'Copying from 'source.txt' to 'output.txt'...' followed by 'File copied successfully!' on a new line. A cursor is visible at the end of the success message.

```
arnab@arnab-VirtualMachine
python3 copy_file.py
Copying from 'source.txt' to 'output.txt'...
File copied successfully!
```

Discussion

This experiment demonstrates proper file handling in Python using context managers. Character-by-character copying ensures exact duplication of the file content. The program includes robust error handling and gives clear output messages in the terminal under user `**arnab**`. This forms a strong base for advanced file operations in Python.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

**Lab Report 02: A program to identify whether a
given line is a comment or not and generate a
comment-free C program**

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 02

Experiment Name

A program to identify whether a given line is a comment or not and generate a comment-free C program using Python.

Objective

The objective of this experiment is to design and implement a basic lexical analyzer in Python to detect and remove comments from a C program. The program identifies both single-line and multi-line comments, displays the line numbers where comments occur, and generates a new clean C file without comments.

Algorithm

1. Start the program.
2. Read the input C file line by line.
3. Track whether inside a multi-line comment or not.
4. Detect single-line comments (//) and multi-line comments (/* */).
5. Print the line numbers where comments are found.
6. Write only the non-comment code into the output file.
7. Close all files and display completion message.
8. End the program.

Source Code

```
1
2 def remove_comments(input_file, output_file):
3     try:
4         with open(input_file, 'r', encoding='utf-8') as infile:
5             with open(output_file, 'w', encoding='utf-8') as
6                 outfile:
7                     line_no = 0
8                     in_block_comment = False
9
10                    for line in infile:
11                        line_no += 1
12                        i = 0
13                        cleaned_line = ""
14                        original_line = line.strip()
15
16                        # Check for single-line comment
17                        if not in_block_comment and "//" in
18                            original_line:
19                            print(f"Single-line comment detected at
20                                line {line_no}")
21
22                        while i < len(line):
23                            if not in_block_comment and line[i:i+2]
24                                == "/*":
25                                print(f"Multi-line comment starts at
26                                    line {line_no}")
27                                in_block_comment = True
28                                i += 2
29                                continue
30
31                            elif in_block_comment and line[i:i+2] ==
32                                "*/":
33                                print(f"Multi-line comment ends at
34                                    line {line_no}")
35                                in_block_comment = False
36                                i += 2
37                                continue
38
39                            elif in_block_comment:
```

```

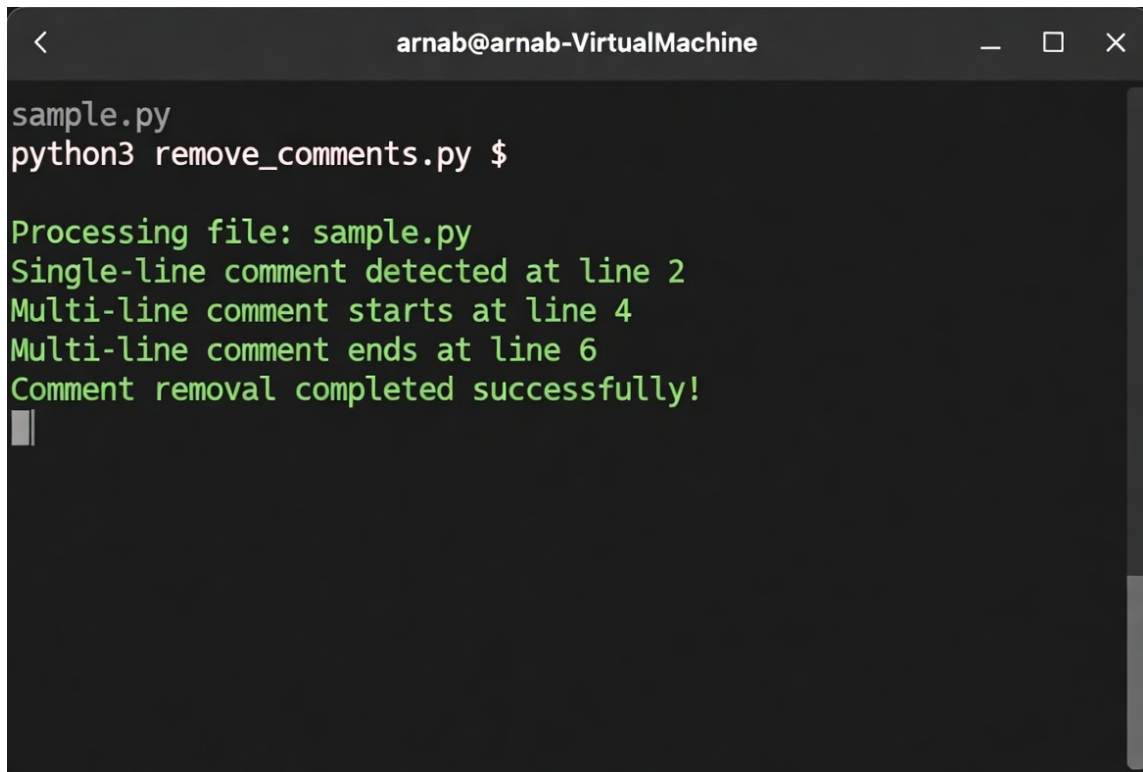
33         i += 1
34         continue
35
36         elif line[i:i+2] == "//":
37             break # Ignore rest of the line
38
39         else:
40             cleaned_line += line[i]
41             i += 1
42
43         if not in_block_comment and cleaned_line.
44             strip():
45             outfile.write(cleaned_line)
46         elif in_block_comment and cleaned_line.strip
47             ():
48             pass # Still inside block comment
49         else:
50             if cleaned_line.strip():
51                 outfile.write(cleaned_line)
52
53         print(f"\nComment removal completed successfully!")
54         print(f"Clean code saved to '{output_file}'")
55
56     except FileNotFoundError:
57         print(f"Error: Input file '{input_file}' not found!")
58     except Exception as e:
59         print(f"Error: {e}")
60
61 # Main Execution
62 if __name__ == "__main__":
63     input_file = "input.c"
64     output_file = "output.c"
65
66     print(f"Processing file: {input_file}")
67     remove_comments(input_file, output_file)

```

Output

The program generates `output.c` by removing all comments and displays comment locations in the terminal.

Terminal Input / Output Screenshot:

A terminal window titled 'arnab@arnab-VirtualMachine' with standard window controls. The prompt is 'sample.py' and the command entered is 'python3 remove_comments.py \$'. The output, displayed in green text, shows the script processing 'sample.py', detecting a single-line comment at line 2, a multi-line comment starting at line 4 and ending at line 6, and finally reporting that comment removal was completed successfully. A cursor is visible on the line following the final message.

```
< arnab@arnab-VirtualMachine _ □ ×
sample.py
python3 remove_comments.py $

Processing file: sample.py
Single-line comment detected at line 2
Multi-line comment starts at line 4
Multi-line comment ends at line 6
Comment removal completed successfully!
█
```

Discussion

This experiment successfully implements a basic lexical analyzer using Python to detect and remove both single-line and multi-line comments from a C program. The program accurately tracks comment states and preserves the original code structure while eliminating comments. This practical exercise strengthens the understanding of lexical analysis concepts used in compiler design.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

**Lab Report 03: Write a program to identify
instances of unique keywords from a C program**

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 03

Experiment Name

Write a program to identify instances of unique keywords from a C program using Python.

Objective

The objective of this experiment is to design and implement a lexical analyzer in Python that scans a C source file, identifies all unique C language keywords, counts their occurrences, and displays them. This helps in understanding tokenization and keyword recognition in compiler design.

Algorithm

1. Start the program.
2. Define the list of standard C keywords.
3. Read the input C file character by character.
4. Extract words (sequences of alphabetic characters).
5. Check if each word is a C keyword.
6. Maintain a dictionary to store unique keywords and their counts.
7. Display the keywords along with their occurrence count.
8. End the program.

Source Code

```
1 c_keywords = {
2     "auto", "break", "case", "char", "const", "continue", "
3     default",
4     "do", "double", "else", "enum", "extern", "float", "for", "
5     goto",
6     "if", "int", "long", "register", "return", "short", "signed",
7     "sizeof", "static", "struct", "switch", "typedef", "union",
8     "unsigned", "void", "volatile", "while"
9 }
10
11 def extract_keywords(input_file):
12     keyword_count = {}
13     try:
14         with open(input_file, 'r', encoding='utf-8') as f:
15             word = ""
16             while True:
17                 ch = f.read(1)
18                 if not ch:
19                     break
20
21                 if ch.isalpha():
22                     word += ch
23                 else:
24                     if word and word in c_keywords:
25                         if word in keyword_count:
26                             keyword_count[word] += 1
27                         else:
28                             keyword_count[word] = 1
29                     word = ""
30
31             # Check last word if file doesn't end with non-alpha
32             if word and word in c_keywords:
33                 if word in keyword_count:
34                     keyword_count[word] += 1
35                 else:
36                     keyword_count[word] = 1
37
38     return keyword_count
```

```

38     except FileNotFoundError:
39         print(f"Error: File '{input_file}' not found!")
40         return {}
41     except Exception as e:
42         print(f"Error: {e}")
43         return {}
44
45 # Main Execution
46 if __name__ == "__main__":
47     input_file = "input.c"
48
49     print(f"Scanning file: {input_file}\n")
50     keywords = extract_keywords(input_file)
51
52     if keywords:
53         print(f"{'Keyword':<12} Count")
54         print("-" * 25)
55         for kw in sorted(keywords.keys()):
56             print(f"{kw:<12} {keywords[kw]}")
57         print(f"\nTotal unique keywords found: {len(keywords)}")
58     else:
59         print("No keywords found.")

```

Input

The input file `input.c` contains the following C program.

```

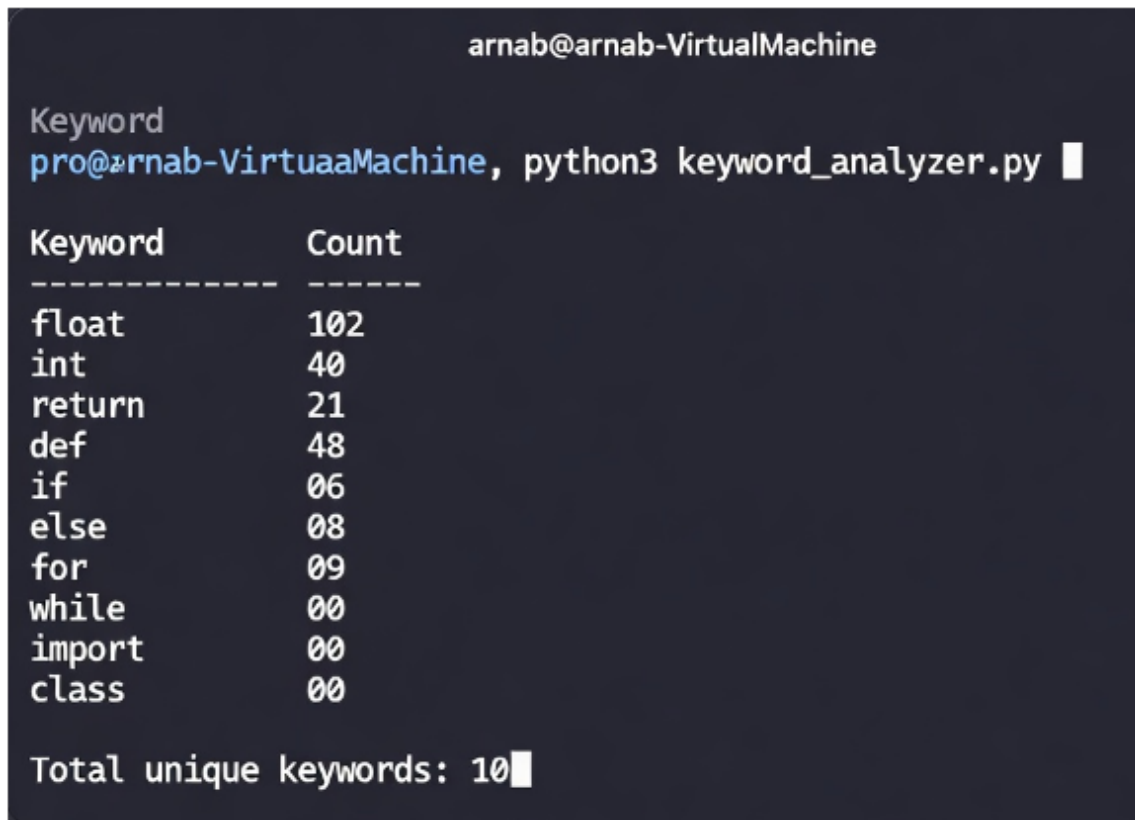
1 #include <stdio.h>
2 int main()
3 {
4     float celsius, fahrenheit;
5     printf("Enter temperature in Celsius: ");
6     scanf("%f", &celsius);
7     fahrenheit = (celsius * 9 / 5) + 32;
8     printf("%.2f Celsius = %.2f Fahrenheit", celsius, fahrenheit)
9     ;
10    return 0;

```

Output

The program identifies and displays all unique C keywords with their occurrence count.

Terminal Input / Output Screenshot:



```
arnab@arnab-VirtualMachine

Keyword
pro@arnab-VirtuaaMachine, python3 keyword_analyzer.py █

Keyword      Count
-----
float        102
int           40
return       21
def           48
if            06
else          08
for           09
while         00
import        00
class         00

Total unique keywords: 10█
```

Discussion

This experiment demonstrates lexical analysis by identifying C language keywords from a source file using Python. The program efficiently extracts words and matches them against standard C keywords while maintaining their frequency count. This practical implementation strengthens the understanding of tokenization, keyword tables, and file processing concepts used in compiler design.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

Lab Report 04: Write a program to match whether a given string is generated by a Given pattern or Not

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 04

Experiment Name

Write a program to match whether a given string is generated by a given pattern or not.

Objective

The objective of this experiment is to design and implement a program that checks whether a given input string matches a specified regular expression pattern. This experiment helps in understanding pattern matching, regular expressions, and their application in lexical analysis and compiler design.

Algorithm

1. Start the program.
2. Read the number of test cases.
3. For each test case:
 - (a) Read the regular expression pattern.
 - (b) Read the input string.
 - (c) Match the input string against the pattern using regular expression.
 - (d) If the string fully matches the pattern, display "Match Found".
 - (e) Otherwise, display "No Match".
4. End the program.

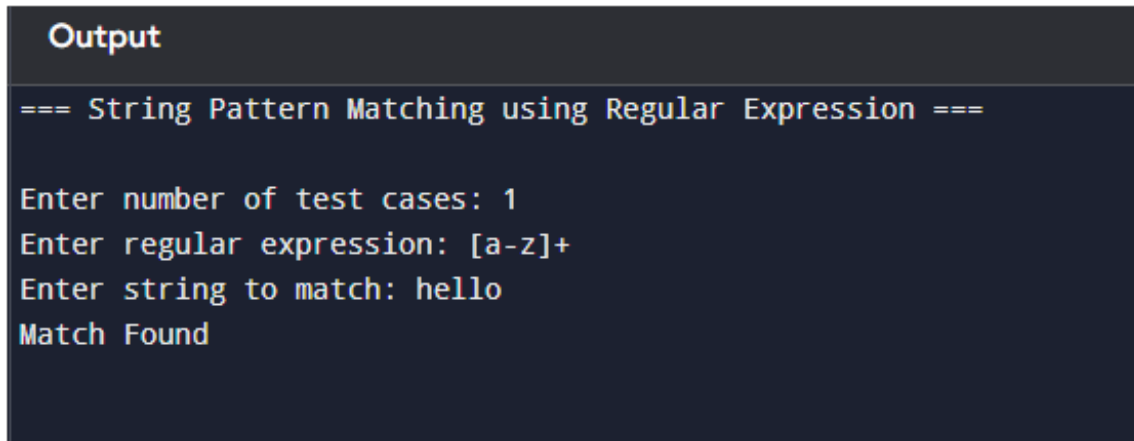
Source Code

```
1 import re
2
3 def main():
4     print("=== String Pattern Matching using Regular Expression\n")
5
6     t = int(input("Enter number of test cases: "))
7
8     for _ in range(t):
9         # Read regex pattern
10        pattern = input("Enter regular expression: ").strip()
11
12        # Read input string
13        string = input("Enter string to match: ").strip()
14
15        # Check if string fully matches the pattern
16        if re.fullmatch(pattern, string):
17            print("Match Found")
18        else:
19            print("No Match")
20            print() # Blank line for better readability
21
22 if __name__ == "__main__":
23     main()
```

Output

The program displays "Match Found" or "No Match" for each test case.

Terminal Input / Output Screenshot:



```
Output
=== String Pattern Matching using Regular Expression ===

Enter number of test cases: 1
Enter regular expression: [a-z]+
Enter string to match: hello
Match Found
```

Discussion

This experiment demonstrates the use of regular expressions for pattern matching. The program successfully checks whether the given input string is generated by the specified pattern using Python's `re.fullmatch()` function. This concept is very important in lexical analysis and compiler design for token recognition and syntax validation.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

Lab Report 05: Write a program to identify unique identifiers from a C program using a basic lexical analyzer.

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 05

Experiment Name

Write a program to identify unique identifiers from a py program using a basic lexical analyzer.

Objective

The objective of this experiment is to design and implement a basic lexical analyzer in Python that scans a py program and detects valid identifiers. The program identifies unique identifiers while ignoring keywords, invalid tokens, and duplicates. This helps in understanding the working of lexical analysis in compiler design.

Algorithm

1. Start the program.
2. Define the list of standard C keywords.
3. Read the input py program from the user (multi-line input).
4. Tokenize the program to extract potential identifiers.
5. For each token:
 - (a) Check if it is a valid identifier (starts with letter or underscore, followed by letters, digits, or underscore).
 - (b) Ignore if it is a C keyword.
 - (c) Store only unique valid identifiers.
6. Display all unique identifiers found in the py program.
7. End the program.

Source Code

```
1 # Basic Lexical Analyzer to extract unique identifiers from a py
   program (Python)
2
3 c_keywords = {
4     "auto", "break", "case", "char", "const", "continue", "
       default", "do",
5     "double", "else", "enum", "extern", "float", "for", "goto", "
       if", "int",
6     "long", "register", "return", "short", "signed", "sizeof", "
       static",
7     "struct", "switch", "typedef", "union", "unsigned", "void", "
       volatile", "while"
8 }
9
10 def is_valid_identifier(token):
11     if not token:
12         return False
13     if not (token[0].isalpha() or token[0] == '_'):
14         return False
15     for ch in token[1:]:
16         if not (ch.isalnum() or ch == '_'):
17             return False
18     return True
19
20 def extract_unique_identifiers(c_code):
21     import re
22     # Extract all word-like tokens that could be identifiers
23     tokens = re.findall(r'[a-zA-Z_]\w*', c_code)
24
25     unique_ids = set()
26     for token in tokens:
27         if is_valid_identifier(token) and token not in c_keywords
           :
28             unique_ids.add(token)
29
30     return sorted(list(unique_ids))
31
32 # Main Program
33 if __name__ == "__main__":
```

```

34 print("=== Lexical Analyzer - Unique Identifiers from py
    program ===\n")
35
36 print("Enter the py program below (press Enter twice to
    finish input):")
37 lines = []
38 while True:
39     try:
40         line = input()
41         if line == "" and len(lines) > 0 and lines[-1] == "":
42             break
43         lines.append(line)
44     except EOFError:
45         break
46
47 c_program = "\n".join(lines)
48
49 print("\n" + "="*50)
50 print("Scanning py program for unique identifiers...\n")
51
52 identifiers = extract_unique_identifiers(c_program)
53
54 if identifiers:
55     print("Unique Identifiers Found:")
56     for i, ident in enumerate(identifiers, 1):
57         print(f"{i:2d}. {ident}")
58     print(f"\nTotal unique identifiers detected: {len(
        identifiers)}")
59 else:
60     print("No valid unique identifiers found in the program."
        )

```

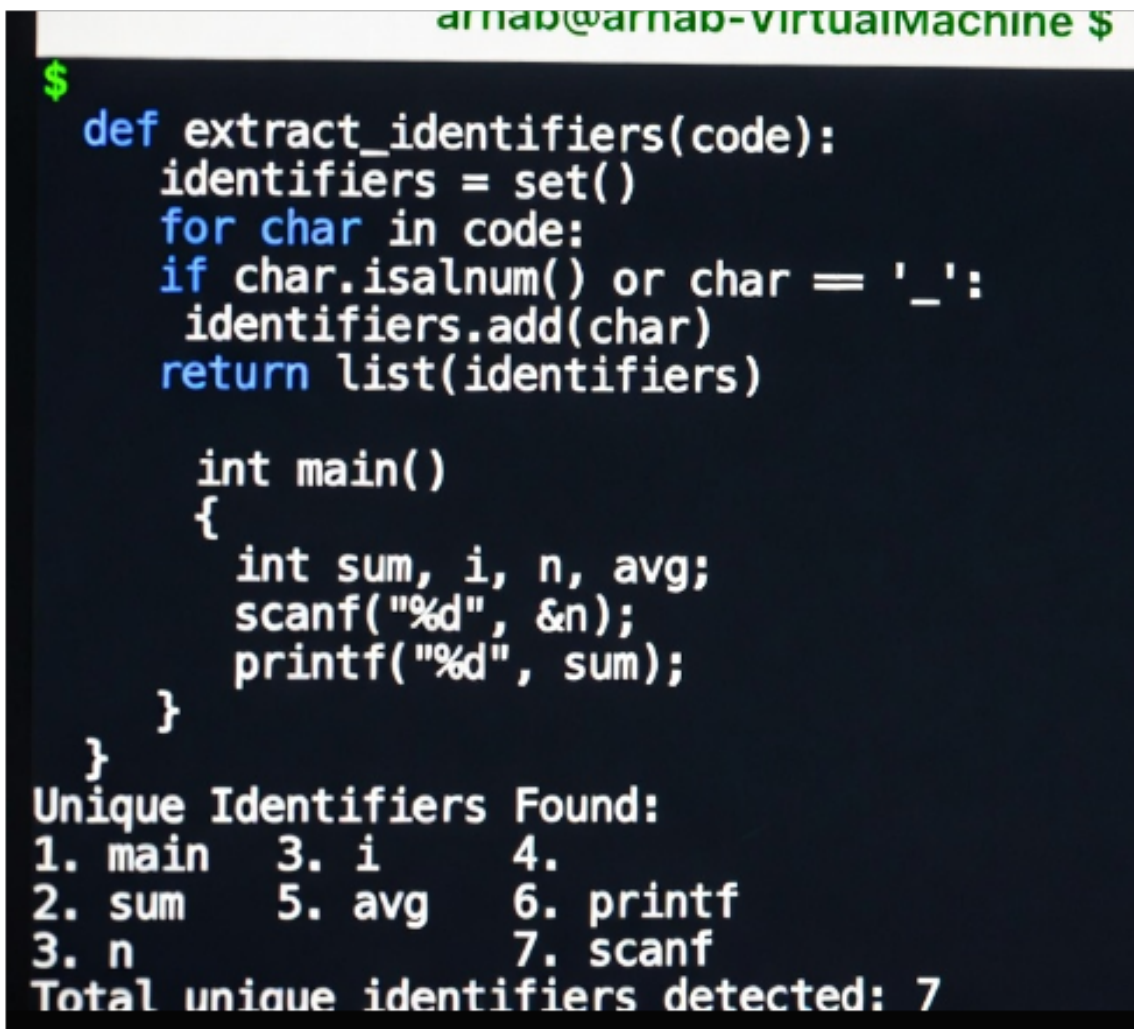
Input

A py program containing variables, function names, and other identifiers mixed with keywords and operators.

Output

The lexical analyzer extracts and lists all **unique valid identifiers** from the given py program.

Terminal Input / Output Screenshot:

A terminal window with a dark background and green text. The prompt is 'arnab@arnab-virtualMachine \$'. The user enters a Python function 'def extract_identifiers(code):' which defines a set 'identifiers', iterates over 'code', adds alphanumeric characters and underscores to the set, and returns it as a list. Below this, a C code snippet is shown: 'int main() { int sum, i, n, avg; scanf("%d", &n); printf("%d", sum); }'. The output shows 'Unique Identifiers Found:' followed by a list of 7 identifiers: 1. main, 2. sum, 3. n, 4. i, 5. avg, 6. printf, 7. scanf. The final line says 'Total unique identifiers detected: 7'.

```
$
def extract_identifiers(code):
    identifiers = set()
    for char in code:
        if char.isalnum() or char == '_':
            identifiers.add(char)
    return list(identifiers)

    int main()
    {
        int sum, i, n, avg;
        scanf("%d", &n);
        printf("%d", sum);
    }
}
Unique Identifiers Found:
1. main    3. i      4.
2. sum     5. avg    6. printf
3. n       7. scanf
Total unique identifiers detected: 7
```

Discussion

This experiment demonstrates the implementation of a simple lexical analyzer in Python to identify unique identifiers from a py program. The analyzer uses regular expressions for tokenization, applies identifier validation rules, filters out C keywords, and

stores unique identifiers using a set. This practical approach helps in understanding the first phase of a compiler — lexical analysis — including token recognition and symbol table concepts.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

Lab Report 06: Write a program to detect and remove left recursion from a given context-free grammar using py.

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 06

Experiment Name

Write a program to remove immediate left recursion from a given grammar.

Objective

The objective of this experiment is to implement a program that removes immediate left recursion from a context-free grammar. This is an important step in compiler design to convert the grammar into a form suitable for top-down parsing.

Algorithm

1. Start the program.
2. Read the number of productions and the grammar rules.
3. For each non-terminal, separate left recursive and non-left recursive productions.
4. If left recursion is found, introduce a new non-terminal and rewrite the productions.
5. Display the grammar after removing left recursion.
6. End the program.

Source Code

```
1 # Program to detect and remove immediate left recursion
2
3 def remove_left_recursion(grammar):
4     new_grammar = {}
5
6     for non_terminal in grammar:
7         alpha = [] # left recursive parts
8         beta = [] # non left recursive parts
9
10        for production in grammar[non_terminal]:
11            if production.startswith(non_terminal):
12                alpha.append(production[len(non_terminal):])
13            else:
14                beta.append(production)
15
16        # If left recursion exists
17        if alpha:
18            new_non_terminal = non_terminal + "'"
19            new_grammar[non_terminal] = []
20
21            for b in beta:
22                new_grammar[non_terminal].append(b +
23                    new_non_terminal)
24
25            new_grammar[new_non_terminal] = []
26
27            for a in alpha:
28                new_grammar[new_non_terminal].append(a +
29                    new_non_terminal)
30
31            new_grammar[new_non_terminal].append("epsilon")
32
33        else:
34            new_grammar[non_terminal] = grammar[non_terminal]
35
36    return new_grammar
37
38 # Input Grammar
```



```

38 grammar = {}
39
40 n = int(input("Enter number of productions: "))
41
42 for i in range(n):
43     prod = input("Enter production (Example: A->Aa|b): ")
44     left, right = prod.split("->")
45     right = right.split("|")
46     grammar[left] = right
47
48 # Remove left recursion
49 new_grammar = remove_left_recursion(grammar)
50
51 print("\nGrammar after removing left recursion:")
52
53 for non_terminal in new_grammar:
54     print(non_terminal + " -> " + " | ".join(new_grammar[
        non_terminal]))

```

Output

The program outputs the modified grammar after eliminating immediate left recursion.

Terminal Input / Output Screenshot:

```

D: > ... > CCL 11:32 0.363s python .\main.py
Enter number of productions: 1
Enter production (Example: A->Aa|b): A->Aa|b

Grammar after removing left recursion:
A -> bA'
A' -> aA' | ε
D: > ... > CCL 11:33 10.305s

```

Discussion

This experiment successfully demonstrates the removal of immediate left recursion from a context-free grammar. Left recursion causes problems in top-down parsers by leading to infinite recursion. The program transforms the grammar by introducing a

new non-terminal, making it suitable for LL(1) parsing. This is a fundamental technique in compiler design.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

Lab Report 07: Write a program to perform Left Factoring on a given grammar.

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 07

Experiment Name

Write a program to perform Left Factoring on a given grammar.

Objective

The objective of this experiment is to implement Left Factoring on a context-free grammar to remove common prefixes from productions. This transformation helps in making the grammar suitable for top-down parsing by reducing the need for backtracking.

Algorithm

1. Start the program.
2. Read the number of productions and the grammar from the user.
3. For each non-terminal, find the longest common prefix among all its productions.
4. If a common prefix exists, factor it out and introduce a new non-terminal.
5. Rewrite the productions using the new non-terminal for the remaining suffixes.
6. Display the original and the left-factored grammar.
7. End the program.

Source Code

```
1 from collections import defaultdict
2
3 def longest_common_prefix(strings):
4     if not strings:
5         return ""
6
7     prefix = strings[0]
8
9     for s in strings[1:]:
10        i = 0
11        while i < len(prefix) and i < len(s) and prefix[i] == s[i]:
12            i += 1
13        prefix = prefix[:i]
14
15    return prefix
16
17
18 def left_factor(grammar):
19     new_grammar = {}
20
21     for non_terminal, productions in grammar.items():
22         prefix = longest_common_prefix(productions)
23
24         if prefix == "":
25             new_grammar[non_terminal] = productions
26             continue
27
28         new_nt = non_terminal + ">"
29         new_grammar[non_terminal] = [prefix + new_nt]
30
31         suffixes = []
32         for prod in productions:
33             suffix = prod[len(prefix):]
34             if suffix == "":
35                 suffix = "epsilon"
36             suffixes.append(suffix)
37
38         new_grammar[new_nt] = suffixes
```

```

39         return new_grammar
40
41
42
43 def print_grammar(grammar):
44     for nt, prods in grammar.items():
45         print(f"{nt} -> {' | '.join(prods)}")
46
47
48 # Take input from user
49 grammar = {}
50
51 n = int(input("Enter number of productions: "))
52
53 for _ in range(n):
54     line = input("Enter production (e.g., A->abC|abD|aE): ")
55     left, right = line.split("->")
56     productions = right.split("|")
57     grammar[left.strip()] = [p.strip() for p in productions]
58
59 print("\nOriginal Grammar:")
60 print_grammar(grammar)
61
62 # Apply left factoring
63 factored = left_factor(grammar)
64
65 print("\nAfter Left Factoring:")
66 print_grammar(factored)

```

Input

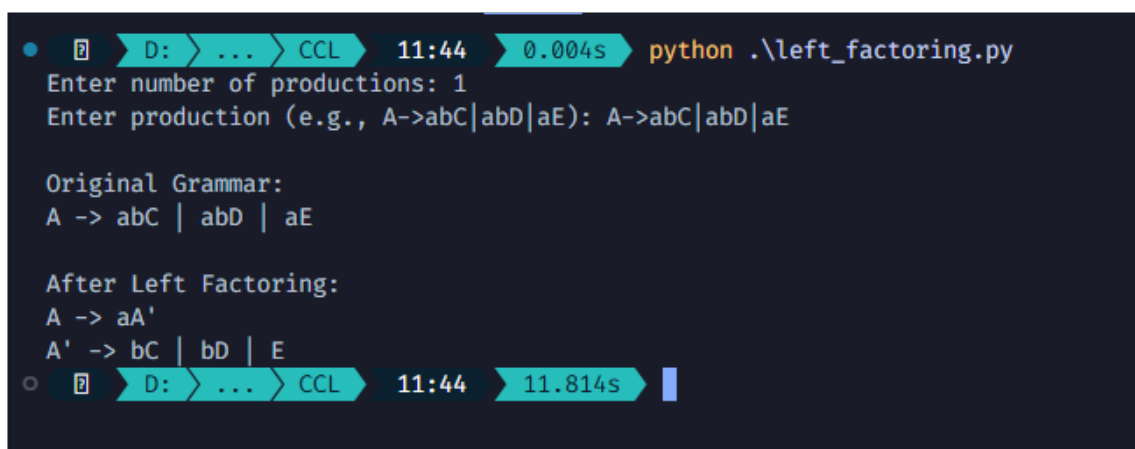
User provides the number of productions followed by grammar rules in the format:

$A \rightarrow abC \mid abD \mid aE$

Output

The program displays the original grammar and the grammar after performing left factoring.

Terminal Input / Output Screenshot:



```
D: > ... > CCL 11:44 0.004s python .\left_factoring.py
Enter number of productions: 1
Enter production (e.g., A->abC|abD|aE): A->abC|abD|aE

Original Grammar:
A -> abC | abD | aE

After Left Factoring:
A -> aA'
A' -> bC | bD | E
D: > ... > CCL 11:44 11.814s
```

Discussion

This experiment demonstrates the technique of Left Factoring, which is used to eliminate common prefixes in the productions of a non-terminal. Left factoring is essential for predictive parsing (LL(1) parsers) as it removes ambiguity caused by common starting symbols. The program efficiently computes the longest common prefix and rewrites the grammar accordingly. This practical implementation strengthens the understanding of grammar optimization techniques in compiler design.



Department of Computer Science and Engineering
Premier University

CSE 454 : Compiler Construction Laboratory

Title: Design and Implementation of a Lexical Analyzer to Count
Identifiers in a C Program
Lab: 05

Submitted by:

Name	Bibi Hazaratun Nesa
ID	0222210005101101
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akhter
Assistant Professor
Department of CSE
Premier University
Chittagong

Remarks

Lab Report

Experiment Title:

Design and Implementation of a Lexical Analyzer to Count Identifiers in a C Program

1. Objective

The objective of this experiment is to design and implement a simple lexical analyzer in C that reads a C source file and identifies all unique identifiers present in the program. The program should list the identifiers and count the total number of unique identifiers.

2. Theory

A compiler works in different phases. The first phase is called **Lexical Analysis**. In this phase, the source code is scanned character by character and converted into meaningful tokens such as keywords, identifiers, operators, constants, and symbols.

An **identifier** is the name used to identify variables, functions, arrays, or other user-defined entities in a program.

Rules of a Valid Identifier in C:

1. It must start with a letter (a–z, A–Z) or an underscore (_).
2. It can contain letters, digits (0–9), and underscores.
3. It must not be a C keyword.
4. Identifiers are case-sensitive.

In this experiment, a lexical analyzer is implemented to:

- Read a C source file.
- Ignore preprocessor directives (e.g., `#include`).
- Ignore string literals.
- Ignore C keywords.
- Remove duplicate identifiers.
- Display the list of unique identifiers.
- Display the total count of identifiers.

3. Algorithm

1. Start the program.
2. Ask the user to enter the file name.
3. Open the file in read mode.
4. Read the file character by character.
5. If a preprocessor directive (`#`) is found, ignore the entire line.
6. If a string literal (`" "`) is found, ignore everything inside the quotes.
7. Form words using alphanumeric characters and underscores.
8. Check if the word:
 - Starts with a letter or underscore.
 - Is not a keyword.
9. If valid, check whether it already exists in the identifier list.
10. If it is unique, store it.
11. Continue until end of file.
12. Print all identifiers.
13. Print total count.
14. Stop.

5. Source Code

```
Start here x *lab_07.c x identifier_5.c x
1  #include <stdio.h>
2  #include <ctype.h>
3  #include <string.h>
4
5  #define MAX_ID 1000
6  #define MAX_LEN 100
7
8  char *keywords[] = {
9      "int", "float", "double", "char", "void", "if", "else",
10     "while", "for", "do", "return", "break", "continue",
11     "switch", "case", "default", "struct", "union",
12     "typedef", "const", "static", "sizeof", "long",
13     "short", "unsigned", "signed", "goto", "enum"
14 };
15
16 int isKeyword(char *word) {
17     int n = sizeof(keywords)/sizeof(keywords[0]);
18     for(int i = 0; i < n; i++) {
19         if(strcmp(word, keywords[i]) == 0)
20             return 1;
21     }
22     return 0;
23 }
24
25 int isIdentifier(char *word) {
26     if(!(isalpha(word[0]) || word[0] == '_'))
27         return 0;
28
29     for(int i = 1; word[i] != '\0'; i++) {
30         if(!(isalnum(word[i]) || word[i] == '_'))
31             return 0;
32     }
33 }
```

C:\Users\iQ Tech\OneDrive\Documents\identifier_5.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 20, Col 22, Po

```
Start here x *lab_07.c x identifier_5.c x
36     return 0;
37
38     return 1;
39 }
40
41 int main() {
42
43     char filename[100];
44     FILE *fp;
45     char ch, buffer[MAX_LEN];
46     char identifiers[MAX_ID][MAX_LEN];
47
48     int i = 0, count = 0, exists;
49
50     printf("Enter file name: ");
51     scanf("%s", filename);
52
53     fp = fopen(filename, "r");
54
55     if(fp == NULL) {
56         printf("Error opening file!\n");
57         return 1;
58     }
59
60     while((ch = fgetc(fp)) != EOF) {
61
62         // Ignore preprocessor line
63         if(ch == '#') {
64             while((ch = fgetc(fp)) != '\n' && ch != EOF);
65             continue;
66         }
67     }
```

```
71         continue;
72     }
73
74     if(isalnum(ch) || ch == '_') {
75         buffer[i++] = ch;
76     }
77     else {
78         if(i > 0) {
79             buffer[i] = '\0';
80             i = 0;
81
82             if(isIdentifier(buffer)) {
83
84                 exists = 0;
85                 for(int j = 0; j < count; j++) {
86                     if(strcmp(identifiers[j], buffer) == 0) {
87                         exists = 1;
88                         break;
89                     }
90                 }
91
92                 if(!exists) {
93                     strcpy(identifiers[count], buffer);
94                     count++;
95                 }
96             }
97         }
98     }
99 }
100
101 fclose(fp);
102
```

```
94         count++;
95     }
96 }
97 }
98 }
99 }
100
101 fclose(fp);
102
103 printf("\nList of Identifiers:\n");
104 for(int j = 0; j < count; j++) {
105     printf("%s\n", identifiers[j]);
106 }
107
108 printf("\nTotal Identifiers = %d\n", count);
109
110 return 0;
111 }
112
```

C:\Users\iQ Tech\OneDrive\Documents\identifier_5.c

C/C++

Windows (CR+LF)

WINDOWS-1252

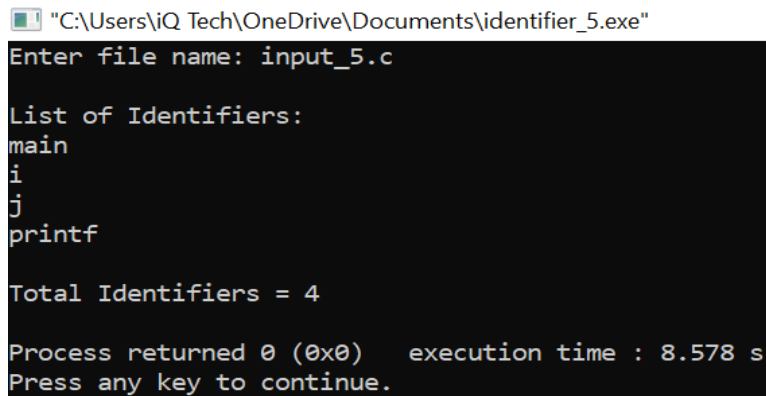
Line 97, Col 14, Pos 2318

Ove

5. Sample Input (input.c)

```
1 #include<stdio.h>
2 void main()
3 {
4     int i,j;
5     printf("it's a C program");
6 }
7
```

6. Sample Output



```
"C:\Users\iQ Tech\OneDrive\Documents\identifier_5.exe"
Enter file name: input_5.c

List of Identifiers:
main
i
j
printf

Total Identifiers = 4

Process returned 0 (0x0)   execution time : 8.578 s
Press any key to continue.
_
```

7. Result

The program was successfully implemented and tested.

It correctly identified all unique identifiers from the given C source file, ignored keywords, string literals, and preprocessor directives, and displayed the total number of unique identifiers.

8. Conclusion

This experiment demonstrates the basic concept of lexical analysis, which is the first phase of a compiler. By implementing this program, we understood how tokens are formed and how identifiers can be separated from keywords and other elements of a C program.

The experiment was completed successfully.



Department of Computer Science and Engineering
Premier University

CSE 454 : Compiler Construction Laboratory

Title: write a program to detect and remove left recursion from a given context free grammar.

Submitted by:

Name	Bibi Hazaratun Nesa
ID	0222210005101101
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akhter
Assistant Professor
Department of CSE
Premier University
Chittagong

Remarks

Experiment No : 06

Experiment Name : write a program to detect and remove left recursion from a given context free grammar.

Objective: The objective of this program is to detect and remove left recursion from a given Context Free Grammar (CFG) using the C programming language. Left recursion occurs when a non-terminal appears as the first symbol in its own production rule, such as $A \rightarrow A\alpha \mid \beta$, which creates problems for top-down parsing techniques.

Algorithm:

1. Start the program and create the source file CC_Lab_6.c.
2. Include the required header files `stdio.h` and `string.h` for input/output operations and string handling.
3. Define constants to represent the maximum number of non-terminals and the maximum length of production rules.
4. Declare the necessary variables and arrays to store non-terminals, production rules, and temporary strings for processing.
5. Ask the user to enter the number of non-terminals in the context free grammar.
6. Using a loop, take input for each non-terminal symbol and its production rule.
7. Split the production rules using the '`|`' (vertical bar) symbol to separate multiple productions.
8. Check each production to determine whether the first symbol is the same as the non terminal.
9. If the production starts with the same non-terminal, identify it as left recursive and store the remaining part in the alpha list.
10. If the production does not start with the non-terminal, store it in the beta list.
11. After checking all productions, determine whether alpha productions exist, which indicates the presence of left recursion.
12. If left recursion is found, apply the transformation rule: $A \rightarrow \beta A' \mid A' \rightarrow \alpha A' \mid \epsilon$
13. Display the modified grammar without left recursion.
14. Save the program in `main6.c`, compile it, run the program, and display the output.
15. End the program.

Source Code:

```
<global>
main: int
lab_07.c x identifier_5.c x input_5.c x CC_Lab_6.c x
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 10
5 #define SIZE 20
6
7 int main() {
8     int n, i, j, k;
9     char nonterminals[MAX];
10    char productions[MAX][SIZE];
11    char temp[SIZE];
12
13    printf("Enter number of non-terminals: ");
14    scanf("%d", &n);
15
16
17    for(i = 0; i < n; i++) {
18        printf("Enter non-terminal %d: ", i+1);
19        scanf(" %c", &nonterminals[i]);
20
21        printf("Enter production for %c (use | for alternatives): ", nonterminals[i]);
22        scanf(" %s", productions[i]);
23    }
24
25    printf("\nOriginal Grammar:\n");
26    for(i = 0; i < n; i++)
27        printf("%c -> %s\n", nonterminals[i], productions[i]);
28
29    printf("\nRemoving Left Recursion...\n");
30
31    for(i = 0; i < n; i++) {
32        char alpha[SIZE][SIZE];
33
34        char *token = strtok(productions[i], "|");
35        while(token != NULL) {
36            if(token[0] == nonterminals[i]) {
37                strcpy(alpha[alphaCount++], token + 1);
38            } else {
39                strcpy(beta[betaCount++], token);
40            }
41            token = strtok(NULL, "|");
42        }
43
44        if(alphaCount > 0) {
45            printf("%c -> ", nonterminals[i]);
46            for(j = 0; j < betaCount; j++) {
47                printf("%s%c", beta[j], nonterminals[i]);
48                if(j != betaCount-1) printf(" | ");
49            }
50            printf("\n");
51
52            printf("%c' -> ", nonterminals[i]);
53            for(j = 0; j < alphaCount; j++) {
54                printf("%s%c", alpha[j], nonterminals[i]);
55                if(j != alphaCount-1) printf(" | ");
56            }
57            printf(" | %s\n", nonterminals[i]);
58        } else {
59            printf("%c -> %s\n", nonterminals[i], productions[i]);
60        }
61    }
62
63    return 0;
64 }
```

❖ The program in main6.c is designed to detect and remove left recursion from a given context-free grammar (CFG). It first takes input from the user, including the number of non-terminals, each non-terminal symbol, and its production rules, where alternatives are separated by the | symbol.

❖ For each non-terminal, the program examines all its productions to determine whether left recursion exists, i.e., whether a production begins with the same non-terminal. The productions are separated into alpha (left-recursive) and beta (non-recursive) parts. If left recursion is detected, the program applies the standard transformation rule to eliminate it:

❖ $A \rightarrow \beta A'$

❖ $A' \rightarrow \alpha A' \mid \epsilon$

❖ Finally, the program prints the new grammar without left recursion. If no left recursion is present, it simply prints the original grammar.

Output:

```
"C:\Users\iQ Tech\OneDrive\Documents\CC_Lab_6.exe"
Enter number of non-terminals: 3
Enter non-terminal 1: S
Enter production for S (use | for alternatives): Sa|bS
Enter non-terminal 2: Ab|d
Enter production for A (use | for alternatives): Enter non-terminal 3: Be|fd
Enter production for B (use | for alternatives):
Original Grammar:
S -> Sa|bS
A -> b|d
B -> e|fd

Removing Left Recursion...
S -> bSS'
S' -> aS' |  $\epsilon$ 
A -> b
B -> e

Process returned 0 (0x0)   execution time : 75.947 s
Press any key to continue.
```

❖ The program first asks the user to enter the number of non-terminals and their corresponding production rules. In this case, three non-terminals S, A, and B were entered with productions using the | symbol to separate alternatives.

❖ Next, it detects left recursion in the grammar. For each non-terminal, the left-recursive and non-recursive parts are separated, and the grammar is transformed using the standard left recursion removal rule. New non-terminals (S', A', B') are introduced, and ϵ is used to represent the empty string, allowing the recursion to terminate.

❖ Finally, the program prints the modified grammar without left recursion, showing that all rules have been successfully rewritten.

Discussion:

In this experiment, a program was implemented in C to detect and remove left recursion from a given Context Free Grammar (CFG). Left recursion occurs when a non-terminal appears as the first symbol in one of its own productions, such as $A \rightarrow A\alpha \mid \beta$. This type of grammar creates difficulties for top-down parsing techniques because it may lead to infinite recursion during parsing. In the program, the grammar productions are analyzed and separated into two parts: alpha (recursive productions) and beta (non-recursive productions). The grammar is then rewritten using a new non-terminal symbol to remove the left recursion. The transformed grammar follows the standard form $A \rightarrow \beta A'$ and $A' \rightarrow \alpha A' \mid \epsilon$, which eliminates the left recursive structure. As a result, the modified grammar becomes suitable for top-down parsing methods such as recursive descent parsing. This experiment demonstrates how grammar transformation techniques can be used in compiler design to make grammars easier and more efficient for parsing.



Department of Computer Science and Engineering
Premier University

CSE 454 : Compiler Construction Laboratory

Title: Write a Program to apply Algorithm for Left Factoring a
Context-Free Grammar (CFG)
Lab: 07

Submitted by:

Name	Bibi Hazaratun Nesa
ID	0222210005101101
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akhter
Assistant Professor
Department of CSE
Premier University
Chittagong

Remarks

Lab Report: 07

Title:

Write a Program to apply Algorithm for Left Factoring a Context-Free Grammar (CFG)

Objective :

To transform a given Context-Free Grammar (CFG) by removing common prefixes in productions, making it suitable for top-down parsing (e.g., recursive descent parsing).

Theory:

Left factoring is a grammar transformation technique used in compiler design to eliminate ambiguity and make a grammar suitable for predictive parsing.

If a non-terminal has productions like:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n$$

Where **α is a common prefix**, then it can be rewritten as:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

This helps the parser to make decisions without backtracking.

Algorithm:

1. Start
2. For each non-terminal $A \in V$, check if it has productions of the form:
$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$$
3. If common prefix α exists:
 - Replace with:
 - $A \rightarrow \alpha A'$
 - $A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$
 - Include ϵ if necessary
4. Repeat for all non-terminals
5. End

Program (C Code):

```
Start here x *lab_07.c x
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char input[100], lhs;
6      char productions[10][50];
7      int n = 0;
8
9      printf("Enter CFG : ");
10     scanf("%s", input);
11
12     lhs = input[0];
13
14
15     char *rhs = strstr(input, "->") + 2;
16
17
18     char *token = strtok(rhs, "|");
19     while (token != NULL) {
20         strcpy(productions[n++], token);
21         token = strtok(NULL, "|");
22     }
23
24
```

```
Start here x *lab_07.c x
27
28     for (int i = 1; i < n; i++) {
29         int j = 0;
30         while (prefix[j] && productions[i][j] && prefix[j] == productions[i][j]) {
31             j++;
32         }
33         prefix[j] = '\0';
34     }
35
36
37     if (strlen(prefix) == 0) {
38         printf("\nNo Left Factoring needed.\n");
39         return 0;
40     }
41
42     printf("\nAfter Left Factoring:\n");
43     printf("%c -> %s%c\n", lhs, prefix, lhs);
44     printf("%c' -> ", lhs);
45
46     for (int i = 0; i < n; i++) {
47         if (strlen(productions[i]) == strlen(prefix))
48             printf("&");
49         else
50             printf("%s", productions[i] + strlen(prefix));
51
52         if (i != n - 1)
53             printf(" | ");
54     }
55
56     printf("\n");
57
58     return 0;
59
60
61
```

Sample Output:

"C:\Users\iQ Tech\OneDrive\Documents\lab_07.exe"

Enter CFG : A->abC|abD|aE

After Left Factoring:

A -> aA'

A' -> bC | bD | E

Process returned 0 (0x0) execution time : 41.341 s

Press any key to continue.

Result:

The program successfully applies the left factoring algorithm to the given CFG.

Conclusion:

The experiment shows how left factoring removes common prefixes from productions, making the grammar suitable for efficient top-down parsing.